Classification

Supervised vs. Unsupervised

fruit	length	width	weight	label
fruit 1	165	38	172	Banana
fruit 2	218	39	230	Banana
fruit 3	76	80	145	Orange
fruit 4	145	35	150	Banana
fruit 5	90	88	160	Orange
...				
fruit n	...	...	...	...

Unsupervised learning:
Learning a model from unlabeled data.

Supervised learning:
Learning a model from labeled data.

Supervised Learning

Unsupervised Learning

Styles of Learning

Supervised	Unsupervised	Semi-Supervised	Reinforcement
• Data has **known labels** or output	• Labels or output unknown • Focus on **finding patterns and gaining insight** from the data	• Labels or output known for **a subset of** data • A blend of supervised and unsupervised learning	• Focus on **making decisions** based on previous experience • Policy-making with feedback
• Insurance underwriting • Fraud detection	• Customer clustering • Association rule mining	• Medical predictions (where tests and expert diagnoses are expensive, and only part of the population receives them)	• Game AI • Complex decision problems • Reward systems

Copyright © SAS Institute Inc. All rights reserved.

Supervised Classification

Training samples are labeled

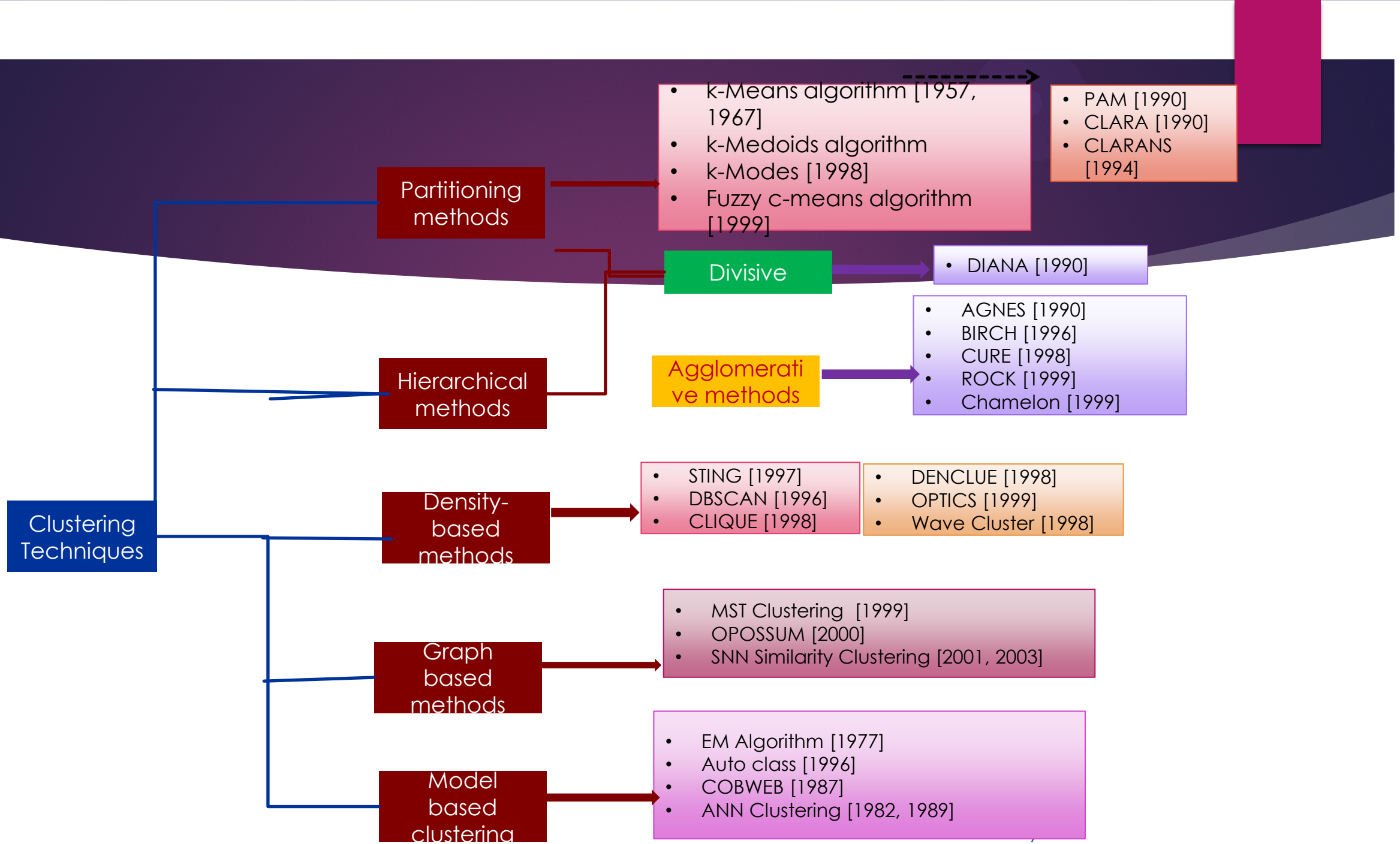
Classification

- Data mining is the process of semi-automatically analyzing large databases to find useful patterns
- **Prediction** based on past history
  - Predict if a credit card applicant poses a good credit risk, based on some attributes (income, job type, age, ..) and past history
  - Predict if a pattern of phone calling card usage is likely to be fraudulent
- Some examples of prediction mechanisms:
  - **Classification**
    - Given a new item whose class is unknown, predict to which class it belongs
  - **Regression** formulae
    - Given a set of mappings for an unknown function, predict the function result for a new parameter value

Unsupervised Learning

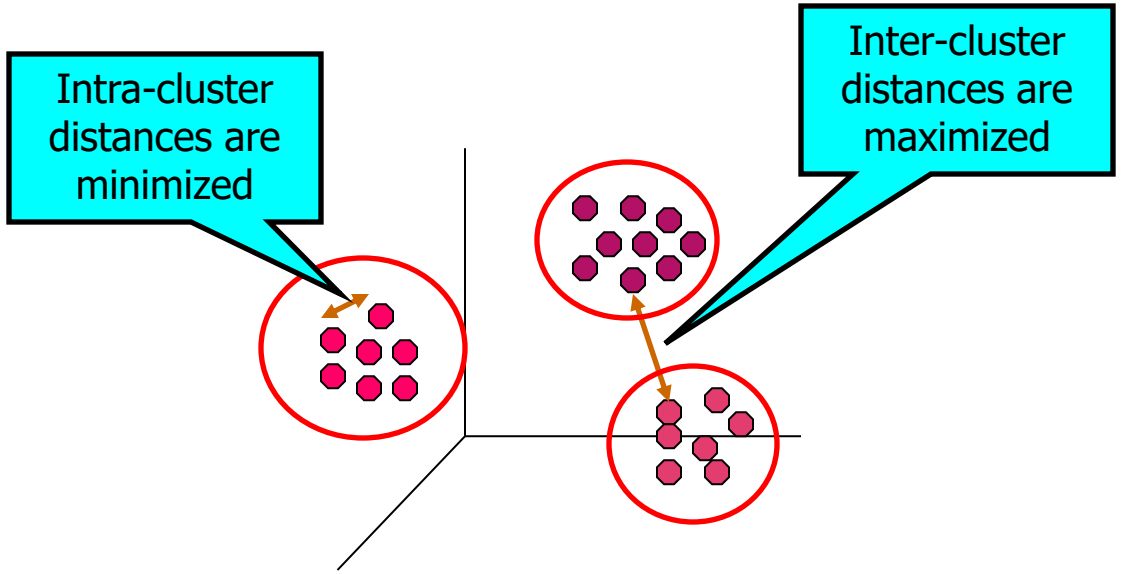
Clustering

- Clustering: Intuitively, finding clusters of points in the given data such that similar points lie in the same cluster
- Can be formalized using distance metrics in several ways
  - Group points into *k* sets (for a given *k*) such that the average distance of points from the centroid of their assigned group is minimized
    - Centroid: point defined by taking average of coordinates in each dimension.
  - Another metric: minimize average distance between every pair of points in a cluster
- Has been studied extensively in statistics, but on small data sets
  - Data mining systems aim at clustering techniques that can handle very large data sets
  - E.g., the Birch clustering algorithm (more shortly)


Clustering Techniques
Partitioning methods
k-Means algorithm [1957, 1967]
k-Medoids algorithm
k-Modes [1998]
Fuzzy c-means algorithm [1999]
PAM [1990]
CLARA [1990]
CLARANS [1994]
Hierarchical methods
Divisive
DIANA [1990]
Agglomerative methods
AGNES [1990]
BIRCH [1996]
CURE [1998]
ROCK [1999]
Chamelon [1999]
Density-based methods
STING [1997]
DBSCAN [1996]
CLIQUE [1998]
DENCLUE [1998]
OPTICS [1999]
Wave Cluster [1998]
Graph based methods
MST Clustering [1999]
OPOSSUM [2000]
SNN Similarity Clustering [2001, 2003]
Model based clustering
EM Algorithm [1977]
Auto class [1996]
COBWEB [1987]
ANN Clustering [1982, 1989]

What is Cluster Analysis?

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



K-Means: A Centroid-Based Technique

Input:

- *k*: the number of clusters,
- *D*: a data set containing *n* objects.

Output: A set of *k* clusters.

Method:

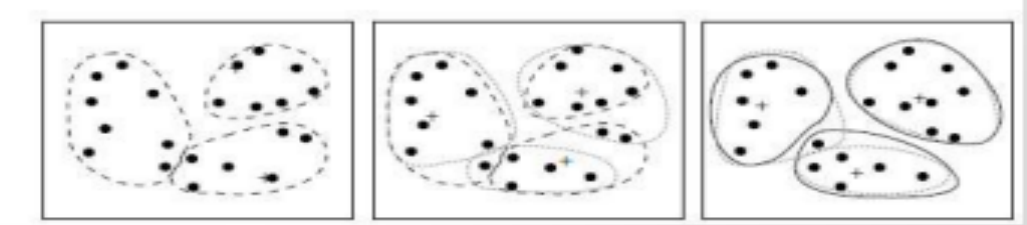
(1) arbitrarily choose *k* objects from *D* as the initial cluster centers;

(2) **repeat**

(3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;

(4) update the cluster means, that is, calculate the mean value of the objects for each cluster;

(5) **until** no change;



Similarity measures

- How to determine similarity between data points
  - using various distance metrics
- Let **x** = $(x_1,\ldots,x_n)$ and **y** = $(y_1,\ldots y_n)$ be n-dimensional vectors of data points of objects g_1 and g_2
  - g_1, g_2 can be two different genes in microarray data
  - n can be the number of samples

Distance measure

- Euclidean distance $$d(g_1, g_2) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$
- Manhattan distance $$d(g_1, g_2) = \sum_{i=1}^{n} \left|(x_i - y_i)\right|$$
- Minkowski distance $$d(g_1, g_2) = \sqrt[m]{\sum_{i=1}^{n} (x_i - y_i)^m}$$

Geometric Distance Measure

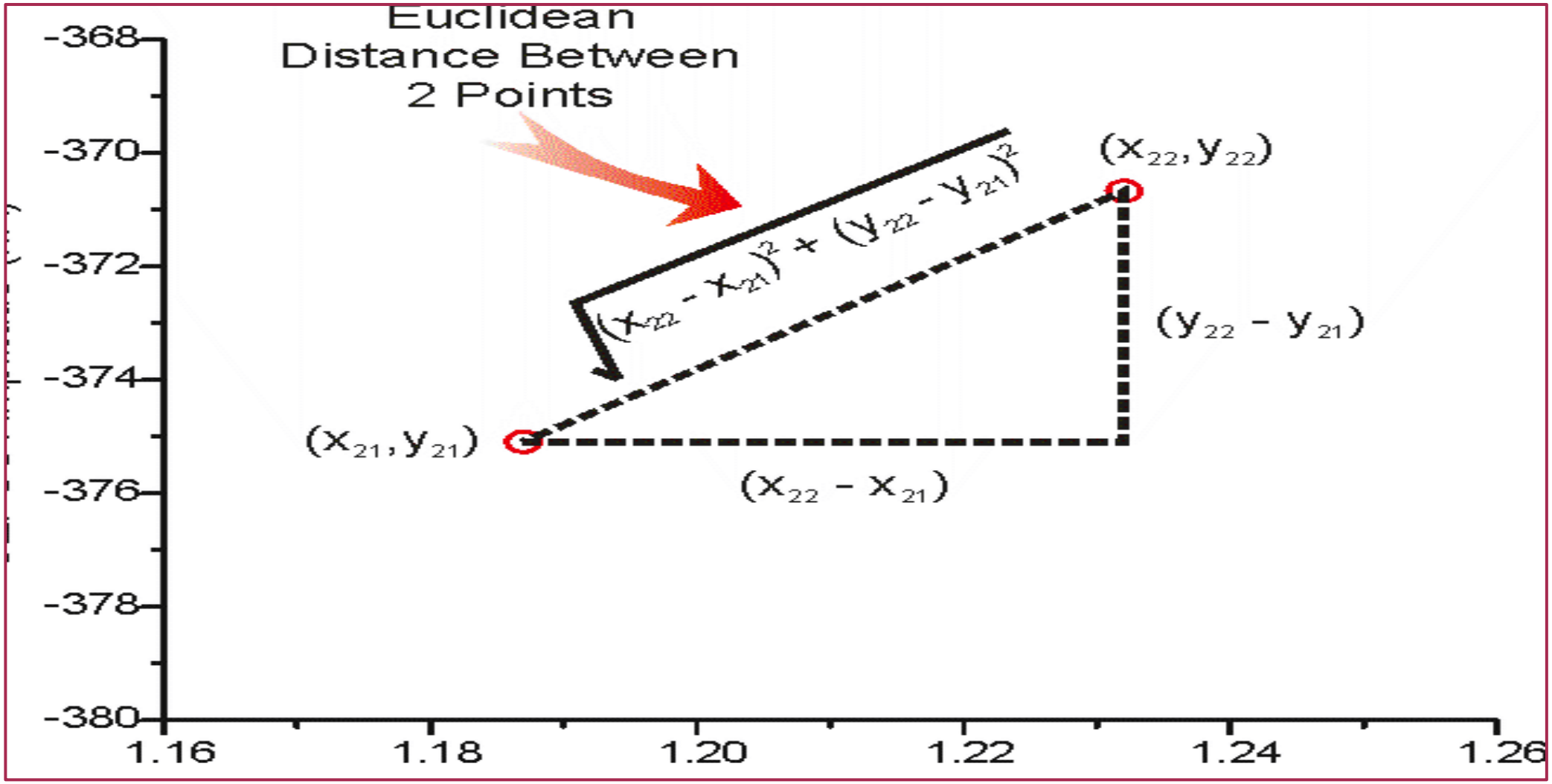
- Geometric distance metrics, primarily, tends to measure the similarity between two or more vectors solely based on the distance between two points in multi-dimensional space.

- The examples of such type of geometric distance measures are Minkowski distance, Euclidean distance and Manhattan distance.

- Minkowski distance is the general form of Euclidean and Manhattan distance. Mathematically, it can be represented as the following:

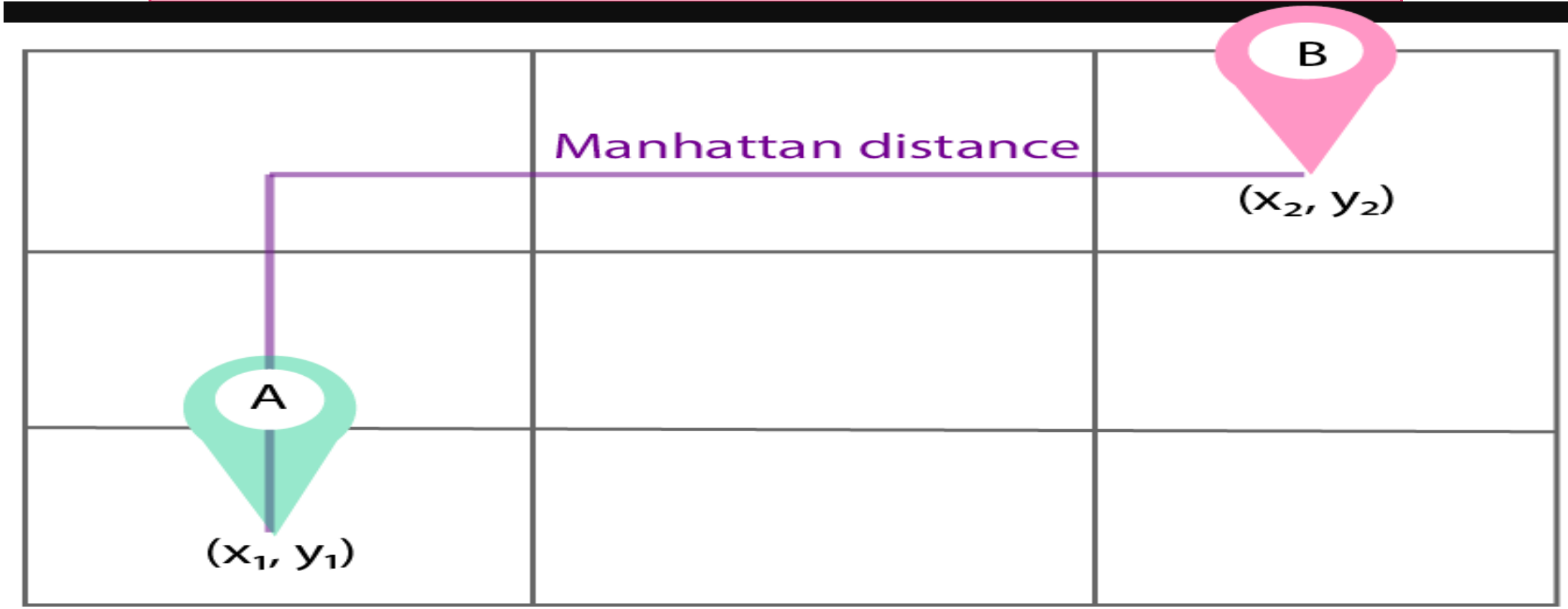
$$D(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{l=1}^{d} |x_{il} - x_{jl}|^{1/p} \right)^p$$

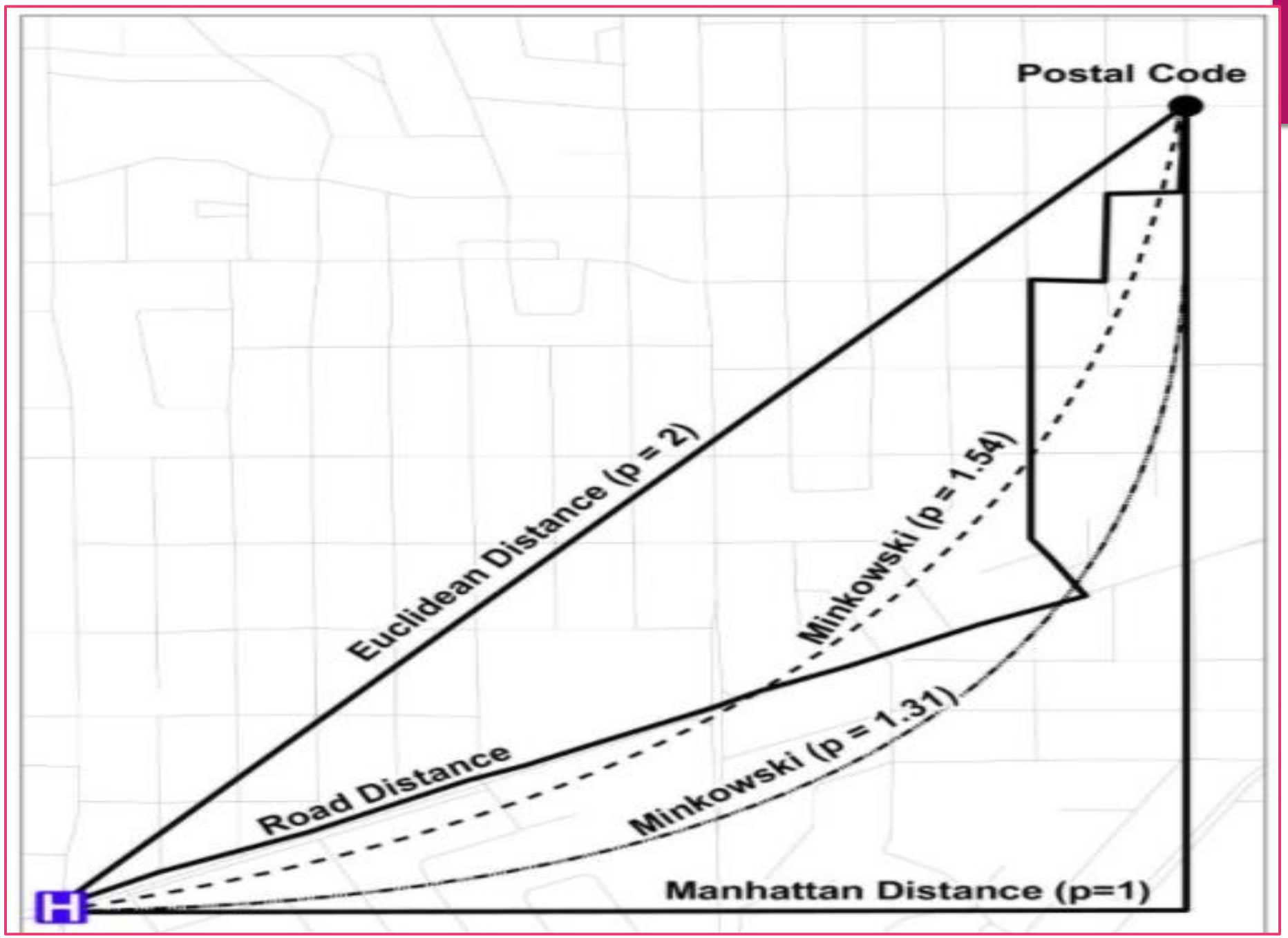
$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$



Manhattan Distance

- If you want to find Manhattan distance between two different points (x1, y1) and (x2, y2) such as the following, it would look like the following:
- Manhattan distance = (x2 – x1) + (y2 – y1)




Postal Code
Euclidean Distance (p = 2)
Minkowski (p = 1.54)
Road Distance
Minkowski (p = 1.31)
Manhattan Distance (p=1)
H

Correlation distance

- Correlation distance

$$r_{xy} = \frac{Cov(X,Y)}{\sqrt{(Var(X) \cdot Var(Y)}}$$

  - Cov(X,Y) stands for covariance of X and Y
    - degree to which two different variables are related
  - Var(X) stands for variance of X
    - measurement of a sample differ from their mean

Correlation distance

- Variance

$$Var(X) = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \overline{X})^2}{n-1}}$$

- Covariance

$$CoVar(X,Y) = \frac{\sum_{i=1}^{n}(x_i - \overline{X})(y_i - \overline{Y})}{n-1}$$

  - Positive covariance
    - two variables vary in the same way
  - Negative covariance
    - one variable might increase when the other decreases
  - Covariance is only suitable for heterogeneous pairs

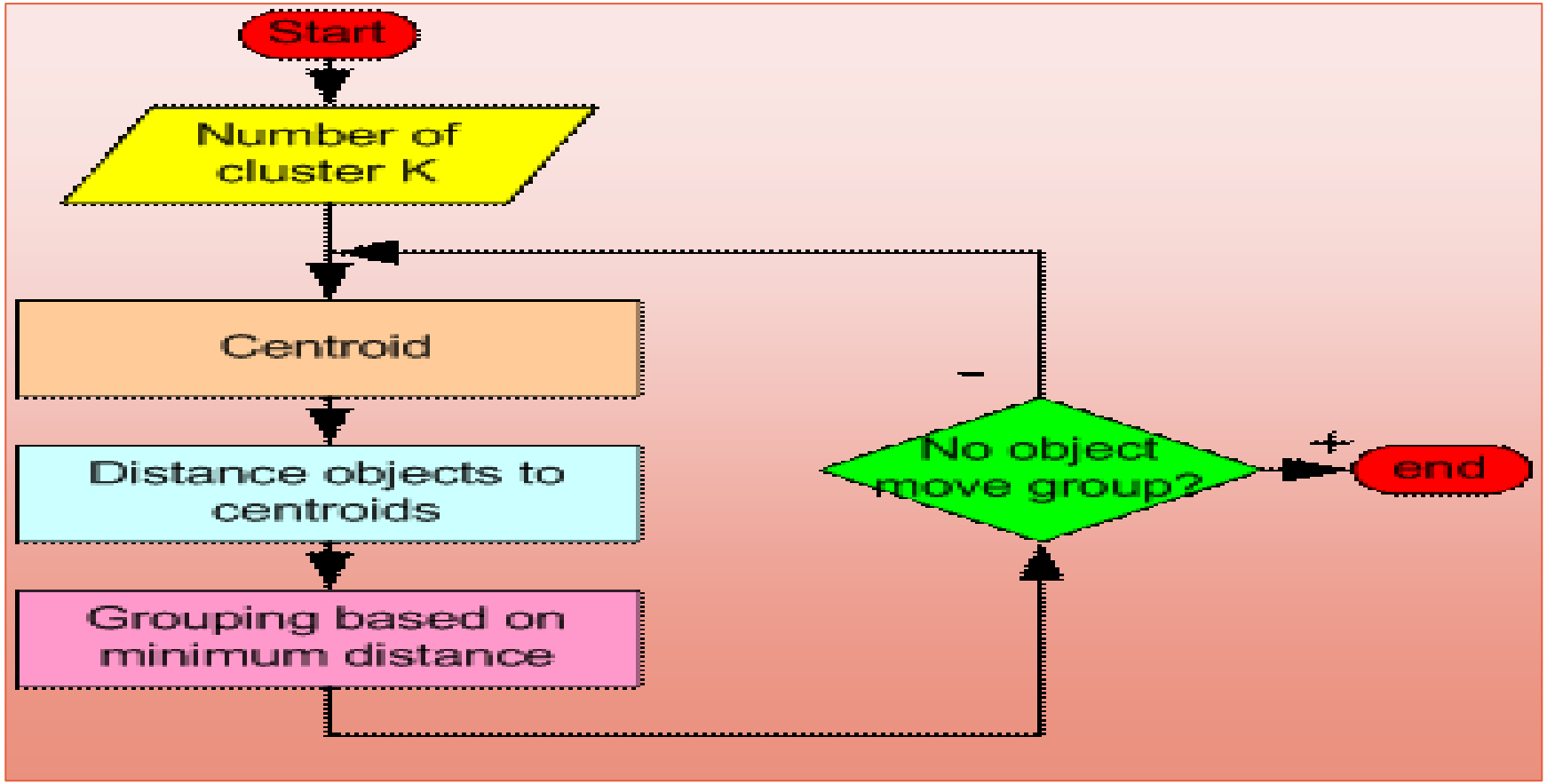
Correlation distance

- Correlation

$$r_{xy} = \frac{Cov(X,Y)}{\sqrt{(Var(X) \cdot Var(Y)}}$$

  - maximum value of 1 if X and Y are perfectly correlated
  - minimum value of 1 if X and Y are exactly opposite
  - $d(X,Y) = 1 - r_{xy}$

How the K-Means Clustering algorithm works?



A Simple example showing the implementation of k-means algorithm (using K=2)

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Rough work

- a=(1,1)

Cent1= (1,1); cent2=(5,7)

Dist(a,cent1)=Sqrt [(1-1)^2 + (1-1)^2] =0

Dist(a,cent2) = Sqrt[(1-5)^2 + (1-7)^2]

=Sqrt[16 + 36]=Sqrt[52]=7.2

Rough work

- B=(1.5,2.0)

Cent1= (1,1); cent2=(5,7)

- Dist(B,Cent1) = sqrt[(1.5-1)^2+ (2.0-1)^2]

  =sqrt[0.5 ^ 2 + 1^2]

  =sqrt[0.25 +1]

  =sqrt[1.25]

  =1.13

Step 1:

Initialization: Randomly we choose following two centroids (k=2) for two clusters.

In this case the 2 centroid are: m1=(1.0,1.0) and m2=(5.0,7.0).

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

	Individual	Mean Vector
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

Step 2:

- Thus, we obtain two clusters containing:

  {1,2,3} and {4,5,6,7}.

- Their new centroids are:

$$m_1 = \left(\frac{1}{3}(1.0+1.5+3.0), \frac{1}{3}(1.0+2.0+4.0)\right) = (1.83, 2.33)$$

$$m_2 = \left(\frac{1}{4}(5.0+3.5+4.5+3.5), \frac{1}{4}(7.0+5.0+5.0+4.5)\right)$$

$$= (4.12, 5.38)$$

Individual	Centroid 1	Centroid 2
1	0	7.21
2 (1.5, 2.0)	1.12	6.10
3	3.61	3.61
4	7.21	0
5	4.72	2.5
6	5.31	2.06
7	4.30	2.92

$$d(m_1, 2) = \sqrt{|1.0-1.5|^2 + |1.0-2.0|^2} = 1.12$$

$$d(m_2, 2) = \sqrt{|5.0-1.5|^2 + |7.0-2.0|^2} = 6.10$$

Step 3:

- Now using these centroids we compute the Euclidean distance of each object, as shown in table.

- Therefore, the new clusters are:

  {1,2} and {**3**,4,5,6,7}

- Next centroids are: m1=(1.25,1.5) and m2 = (3.9,5.1)

Individual	Centroid 1	Centroid 2
1	1.57	5.38
2	0.47	4.28
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

- Step 4 :

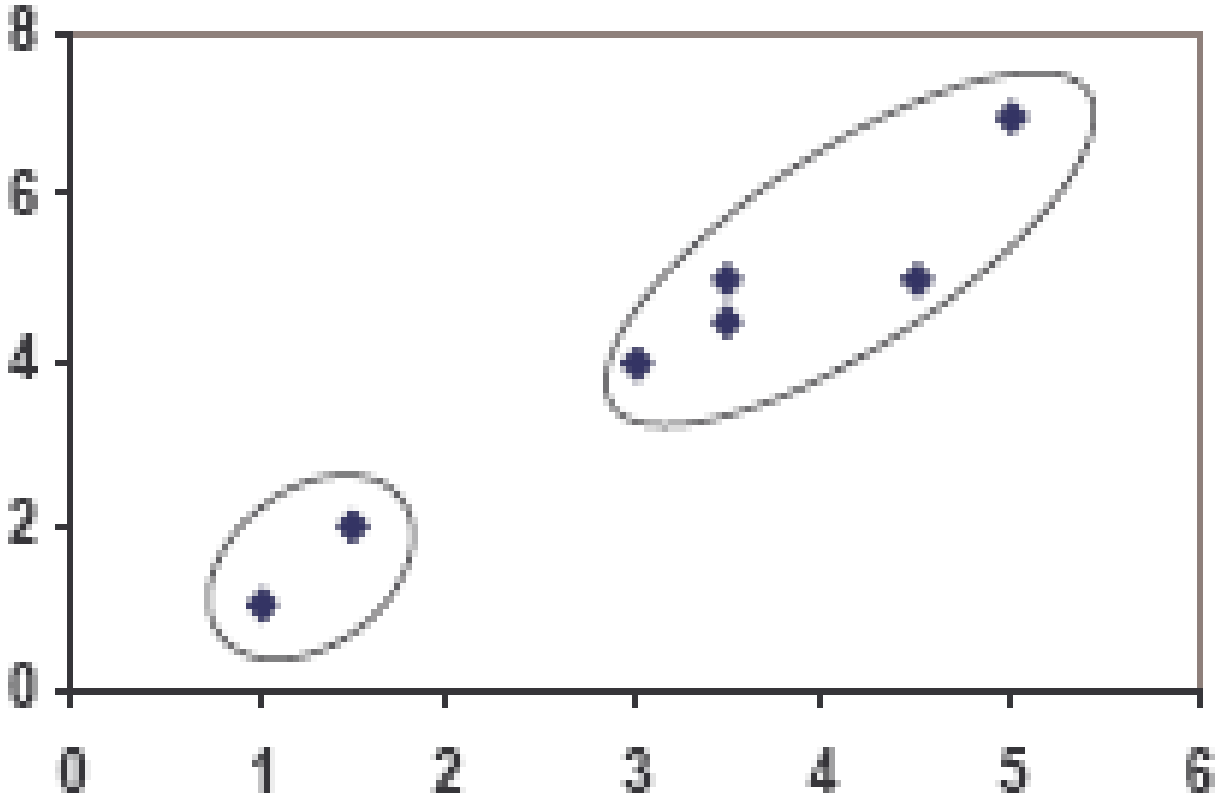
  The clusters obtained are:

  {1,2} and {3,4,5,6,7}

- Therefore, there is no change in the cluster.
- Thus, the algorithm comes to a halt here and final result consist of 2 clusters {1,2} and {3,4,5,6,7}.

Individual	Centroid 1	Centroid 2
1	0.56	5.02
2	0. 56	3.92
3	3.05	1.42
4	6.66	2.20
5	4.16	0.41
6	4.78	0.61
7	3.75	0.72

PLOT



K-means

Data set: $X = \{\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n\}$

Clusters: $C_1, C_2, \cdots C_k$

Codebook : $V = \{\bar{v}_1, \bar{v}_2, \cdots, \bar{v}_k\}$

Partition matrix:

$$\gamma_{ij} = \begin{cases} 1 & \text{if } \bar{x}_j \in C_i \\ 0 & \text{otherwise} \end{cases}$$

$$\Gamma = \{\gamma_{ij}\}$$

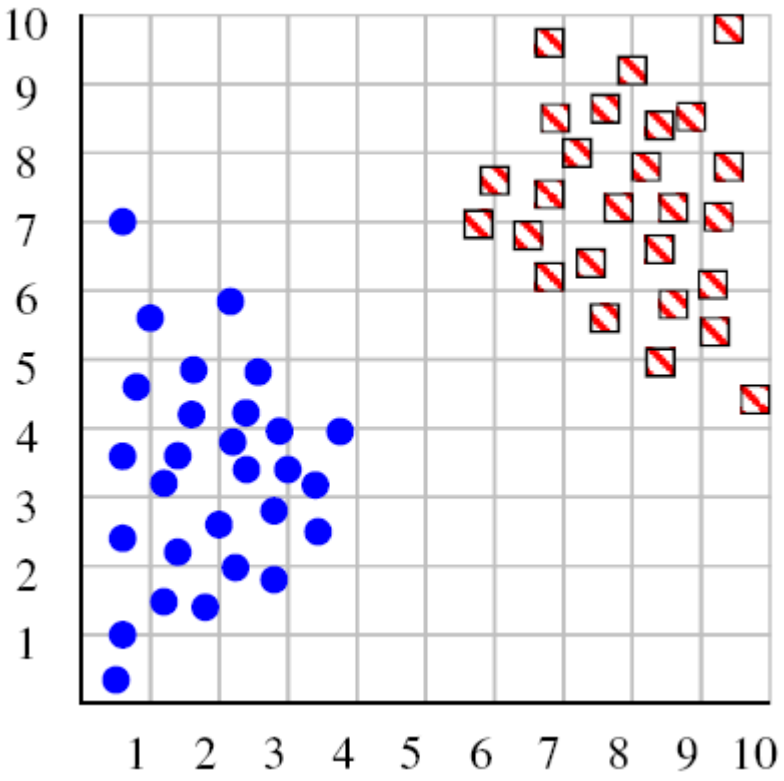
- Minimizes functional:

$$E(\Gamma, V) = \sum_{i=1}^{k} \sum_{j=1}^{n} \gamma_{ij} \left\| \bar{x}_j - \bar{v}_i \right\|^2$$

- Iterative algorithm:
  - Initialize the codebook *V* with vectors randomly picked from *X*
  - Assign each pattern to the nearest cluster
  - Recalculate partition matrix
  - Repeat the above two steps until convergence

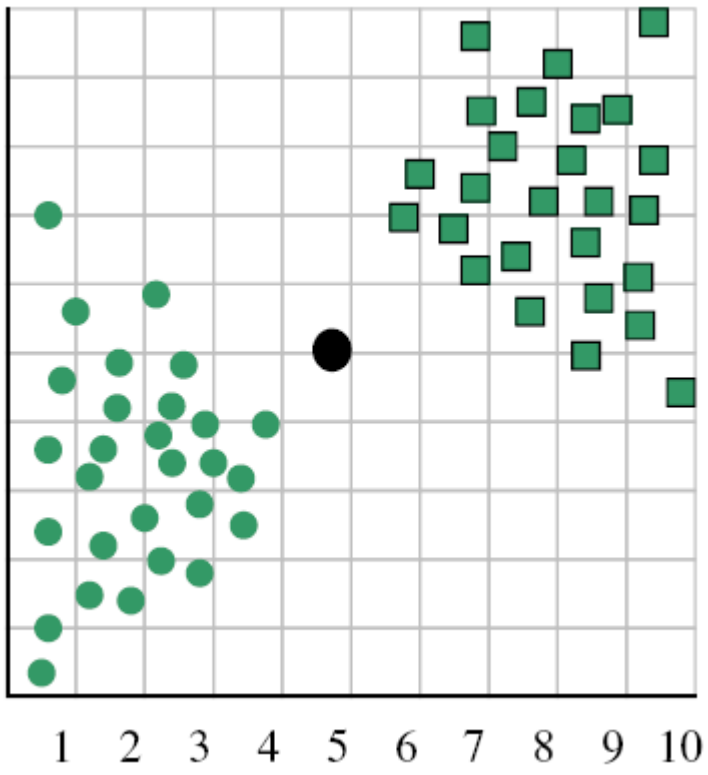
Deciding K

- Try a couple of K



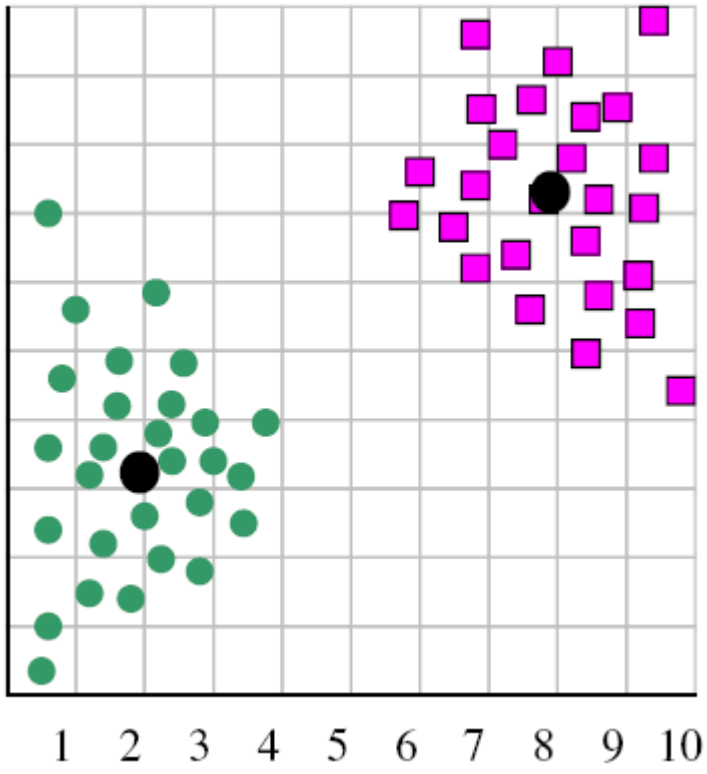
Deciding K

- When k = 1, the objective function is 873.0



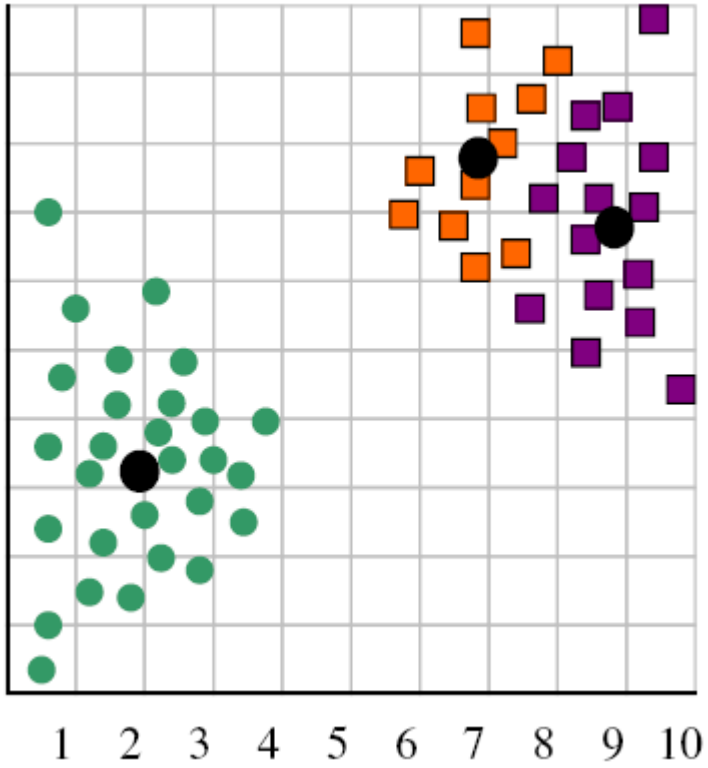
Deciding K

- When k = 2, the objective function is 173.1



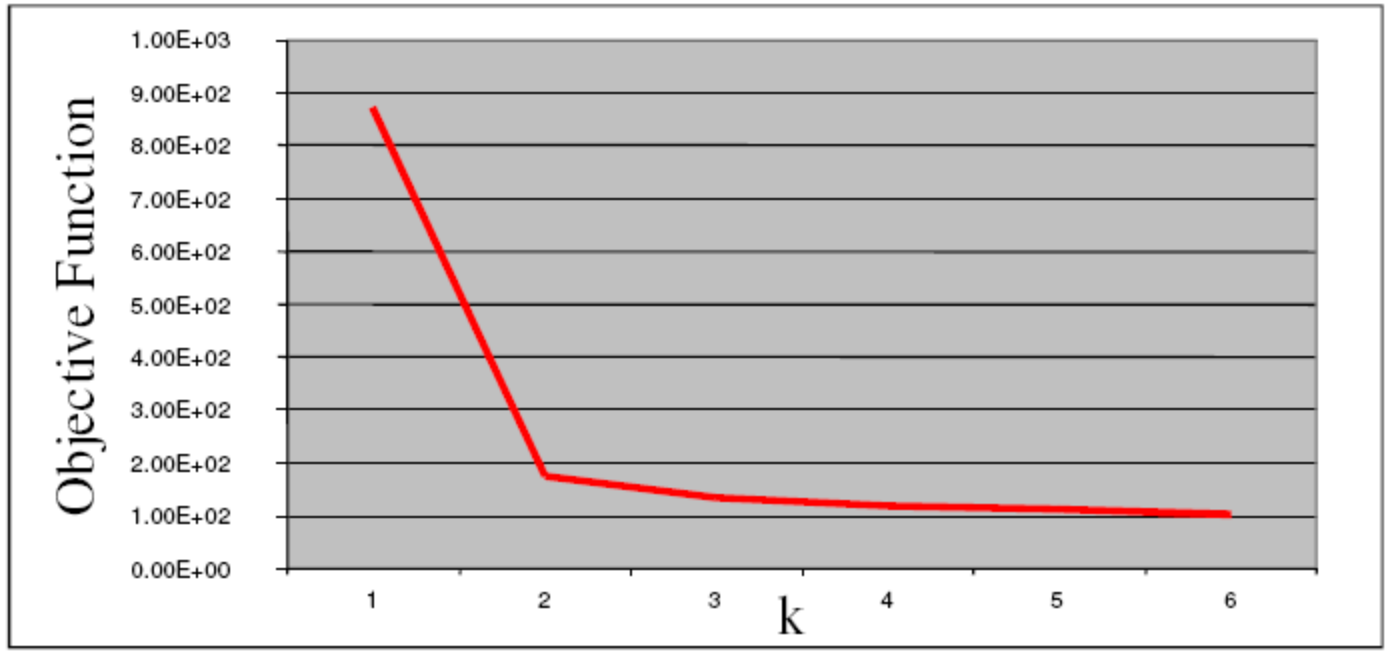
Deciding K

- When k = 3, the objective function is 133.6



Deciding K

- We can plot objective function values for k=1 to 6
- The abrupt change at k=2 is highly suggestive of two clusters
- "knee finding" or "elbow finding"
- Note that the results are not always as clear cut as in this toy example



Back

K-means: Setup

- $x_1, \ldots, x_N$ are data points or vectors of observations
- Each observation (vector x_i) will be assigned to one and only one cluster
- $C(i)$ denotes cluster number for the ith observation
- Dissimilarity measure: Euclidean distance metric
- *K*-means minimizes within-cluster point scatter:

$$W(C) = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\sum_{C(j)=k}\left\|x_i - x_j\right\|^2 = \sum_{k=1}^{K} N_k \sum_{C(i)=k}\left\|x_i - m_k\right\|^2$$

(Exercise)

where

m_k is the mean vector of the kth cluster

N_k is the number of observations in k^{th} cluster

Within and Between Cluster Criteria

Let's consider total point scatter for a set of *N* data points:

$$T = \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} d(x_i, x_j)$$

Distance between two points

T can be re-written as:

$$T = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\left(\sum_{C(j)=k} d(x_i, x_j) + \sum_{C(j)\neq k} d(x_i, x_j)\right)$$
$$= W(C) + B(C)$$

Where,

$$W(C) = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\sum_{C(j)=k} d(x_i, x_j)$$

Within cluster scatter

$$B(C) = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\sum_{C(j)\neq k} d(x_i, x_j)$$

Between cluster scatter

If *d* is square Euclidean distance, then

$$W(C) = \sum_{k=1}^{K} N_k \sum_{C(i)=k} \|x_i - m_k\|^2$$

and $$B(C) = \sum_{k=1}^{K} N_k \|m_k - m\|^2$$

Ex.

Grand mean

Minimizing *W*(*C*) is equivalent to maximizing *B*(*C*)

K-means

- Disadvantages
  - Dependent on initialization
  - Sensitive to outliers (K-medoids)
  - Can deal only with clusters with spherical symmetrical point distribution
  - Deciding *K*

Applications of K-Means Clustering

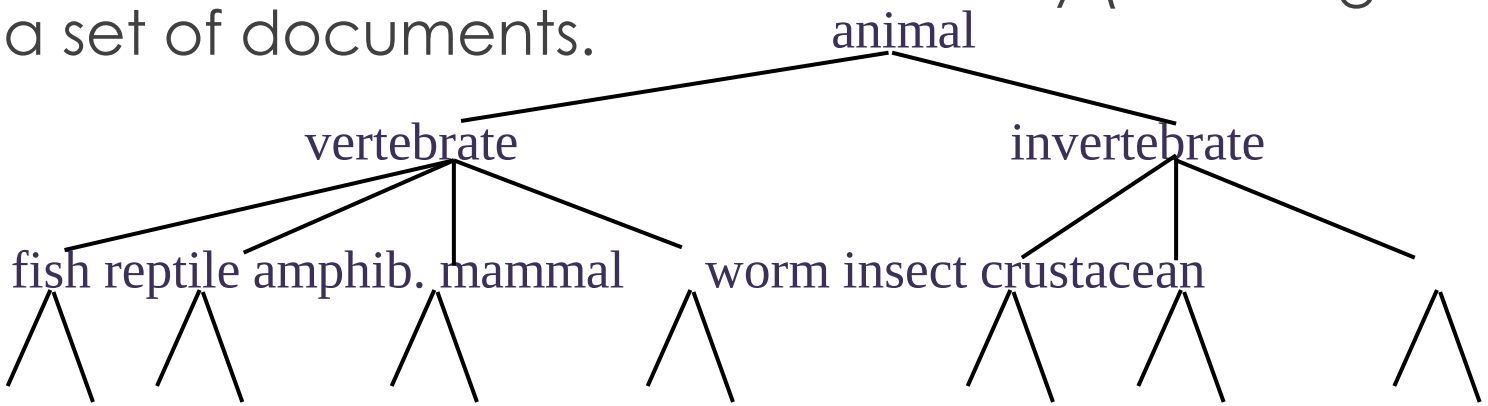
- It is relatively *efficient and fast*. It computes result at **O(tkn)**, where n is number of objects or points, k is number of clusters and t is number of iterations.
- k-means clustering can be applied to *machine learning or data mining*
- *Used on acoustic data in speech understanding to convert waveforms into one of k categories (known as Vector Quantization or Image Segmentation).*
- *Also used for choosing color palettes on old fashioned graphical display devices and Image Quantization.*

K-means: summary

- Algorithmically, very simple to implement
- *K*-means converges, but it finds a local minimum of the cost function
- Works only for numerical observations
- *K* is a user input; alternatively BIC (Bayesian information criterion) or MDL (minimum description length) can be used to estimate *K*
- Outliers can considerable trouble to *K*-means

Hierarchical Clustering

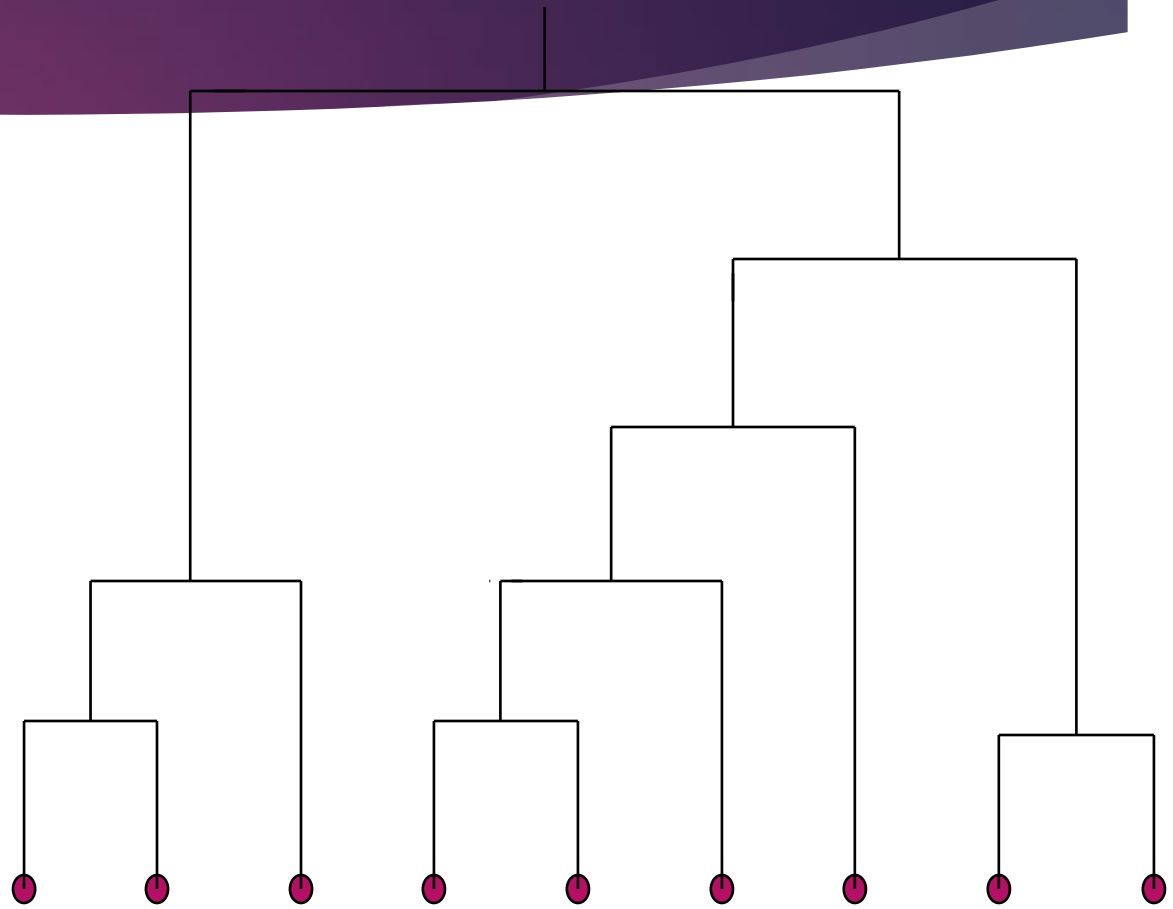
- Build a tree-based hierarchical taxonomy (*dendrogram*) from a set of documents.



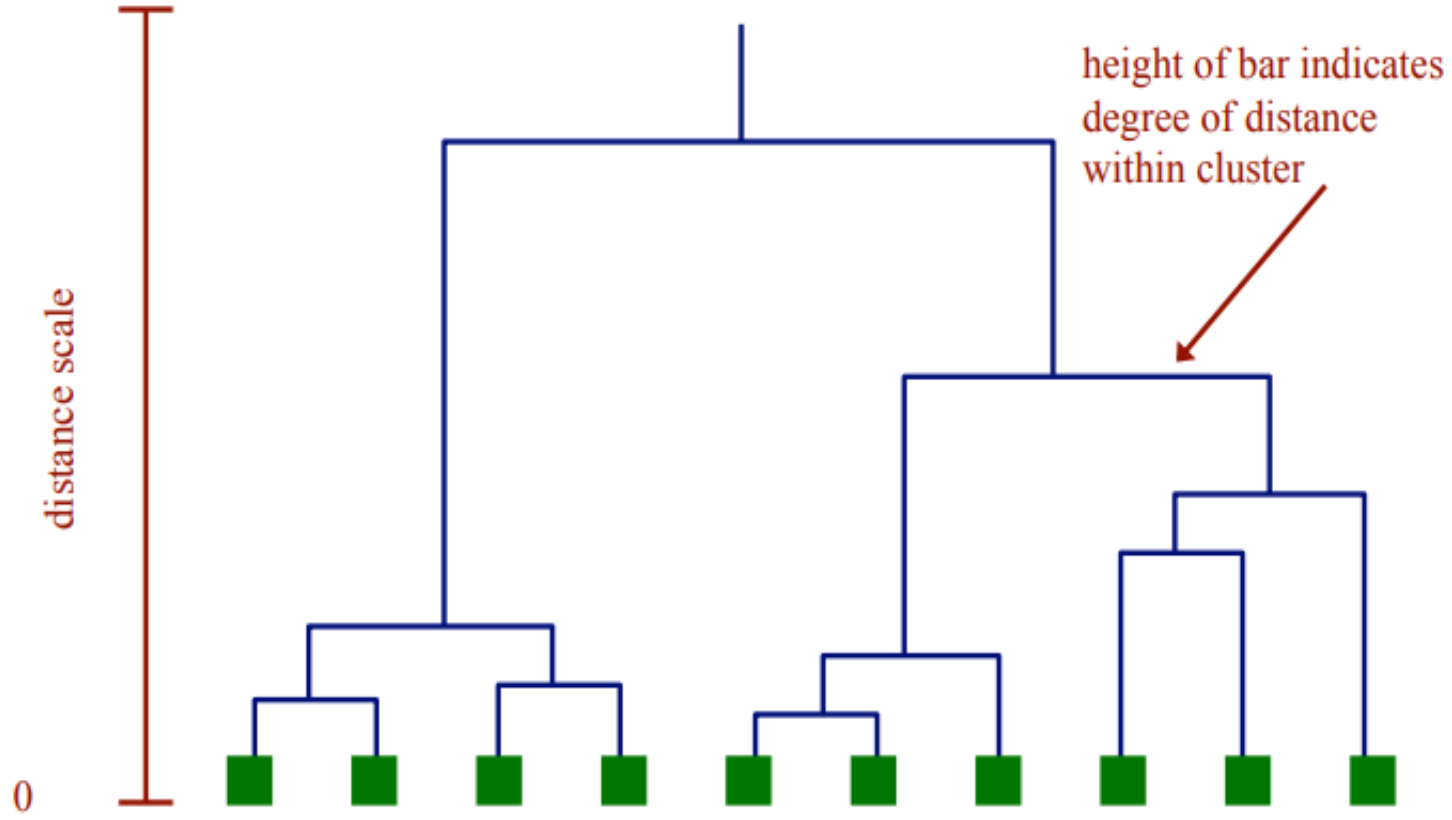
- One approach: recursive application of a partitional clustering algorithm.

Dendrogram: Hierarchical Clustering

- Clustering obtained by cutting the dendrogram at a desired level: each connected component forms a cluster.

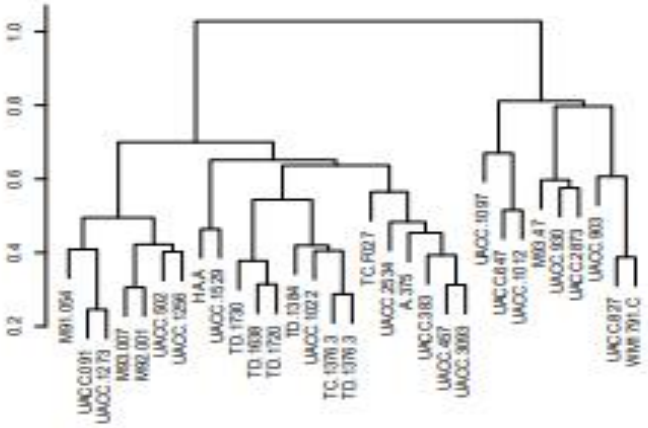


Hierarchical Clustering: A Dendrogram

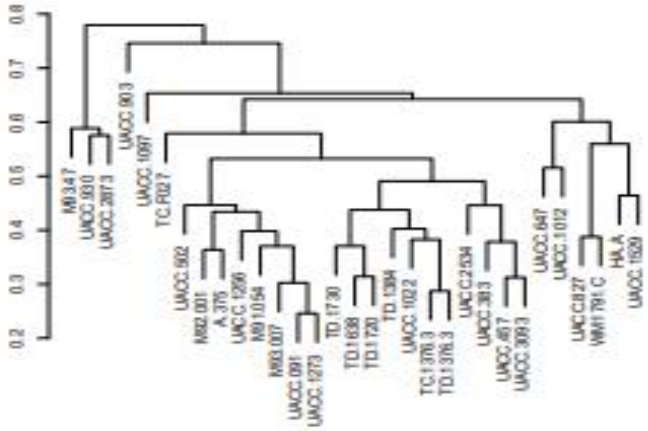


Cluster Analysis

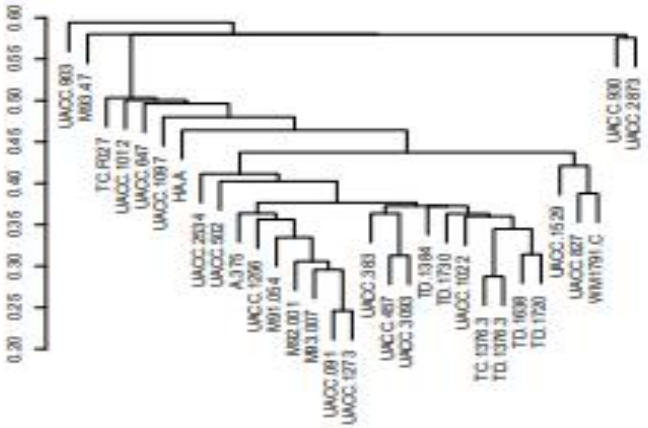
Complete



Average



Single

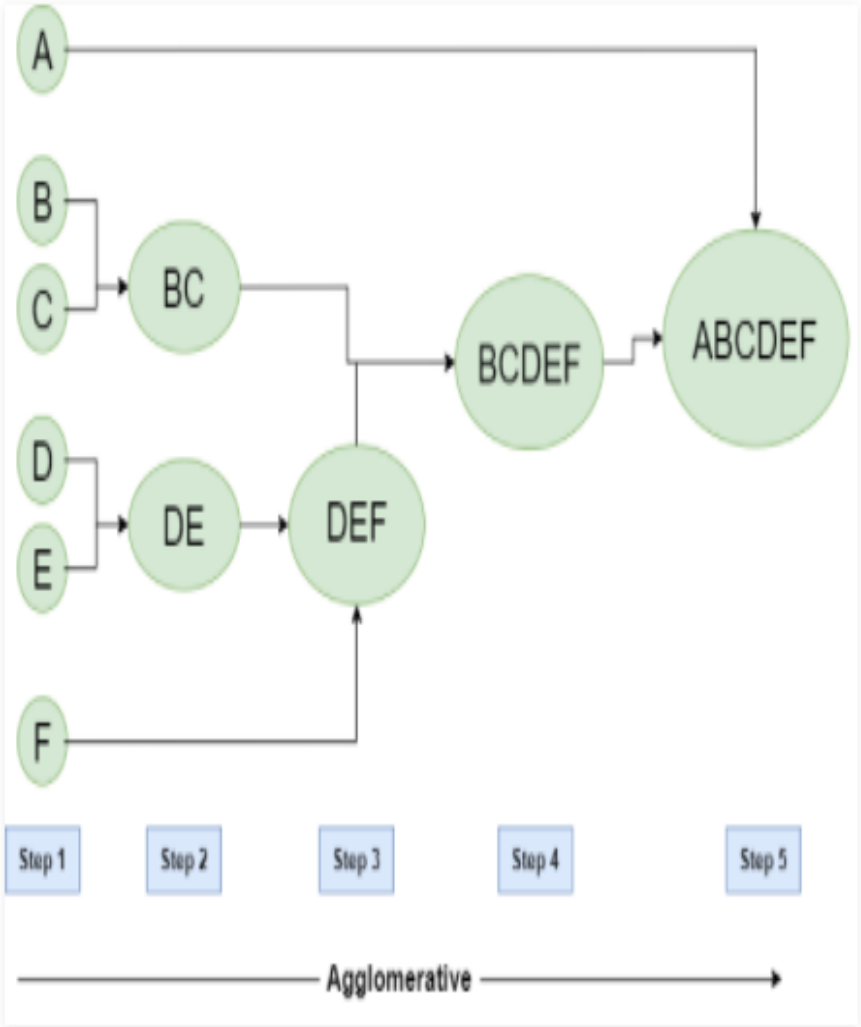


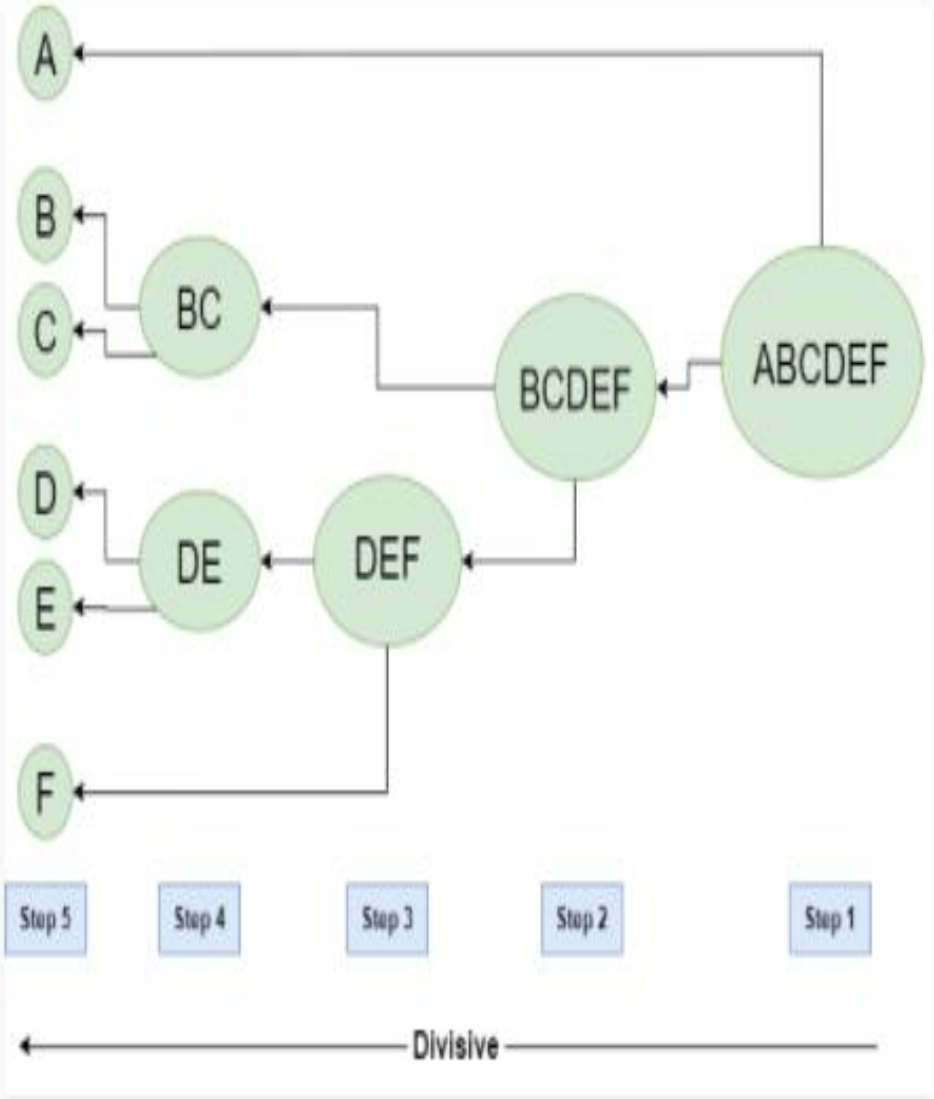
Dendrograms using 3 different linkage methods, 1 - Corr distance (Data from Bittner *et al.*, *Nature*, 2000)

Hierarchical Agglomerative Clustering (HAC)

- Starts with each doc in a separate cluster
  - then repeatedly joins the *closest pair* of clusters, until there is only one cluster.
- The history of merging forms a binary tree or hierarchy.

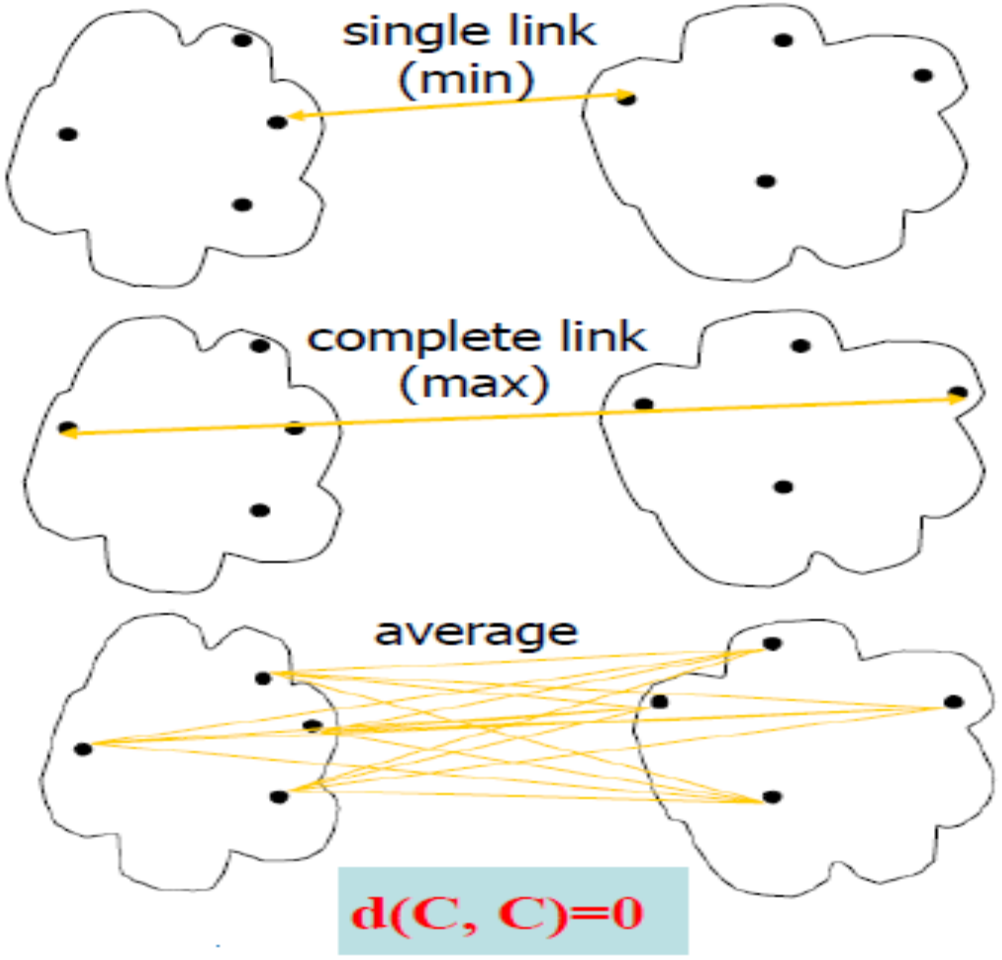
Let's say we have six data points **A, B, C, D, E, F**.





Cluster Distance Measures

- Single link: smallest distance between an element in one cluster and an element in the other, i.e., $d(C_i, C_j) = \min\{d(x_{ip}, x_{jq})\}$
- Complete link: largest distance between an element in one cluster and an element in the other, i.e., $d(C_i, C_j) = \max\{d(x_{ip}, x_{jq})\}$
- Average: avg distance between elements in one cluster and elements in the other, i.e., $d(C_i, C_j) = \text{avg}\{d(x_{ip}, x_{jq})\}$



Cluster Distance Measures

Example: Given a data set of five objects characterised by a single continuous feature, assume that there are two clusters: C_1: {a, b} and C_2: {c, d, e}. (Minkowski distance for distance matrix)

	a	b	c	d	e
Feature	1	2	4	5	6

1. Calculate the distance matrix .

	a	b	c	d	e
a	0	1	3	4	5
b	1	0	2	3	4
c	3	2	0	1	2
d	4	3	1	0	1
e	5	4	2	1	0

2. Calculate three cluster distances between C_1 and C_2.

Single link

$$\mathrm{dist}(C_1, C_2) = \min\{d(a,c), d(a,d), d(a,e), d(b,c), d(b,d), d(b,e)\}$$
$$= \min\{3, 4, 5, 2, 3, 4\} = 2$$

Complete link

$$\mathrm{dist}(C_1, C_2) = \max\{d(a,c), d(a,d), d(a,e), d(b,c), d(b,d), d(b,e)\}$$
$$= \max\{3, 4, 5, 2, 3, 4\} = 5$$

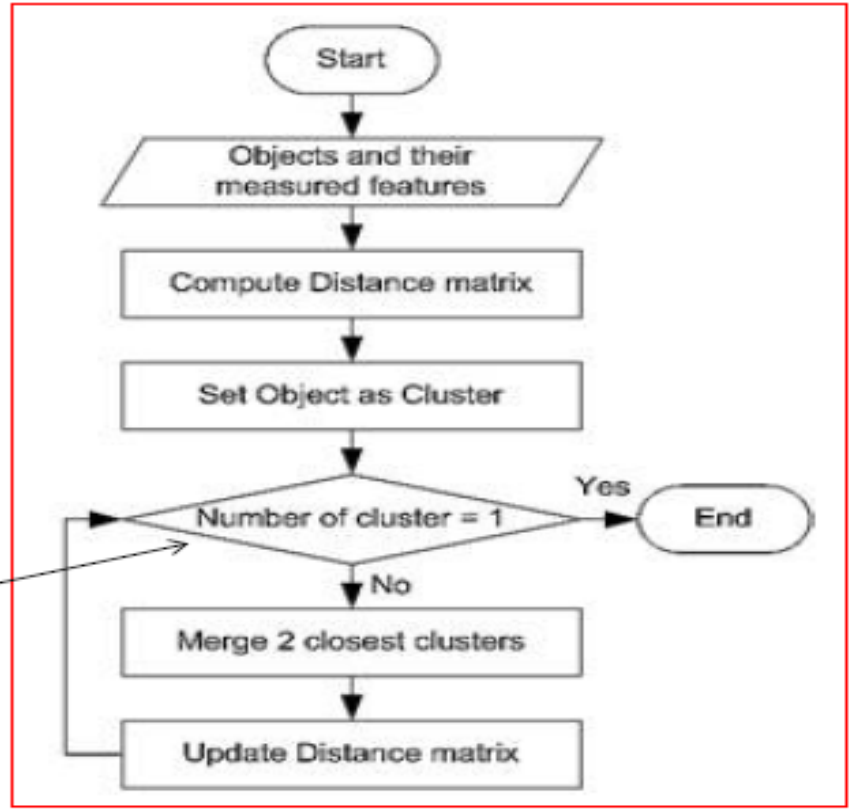
Average

$$\mathrm{dist}(C_1, C_2) = \frac{d(a,c) + d(a,d) + d(a,e) + d(b,c) + d(b,d) + d(b,e)}{6}$$
$$= \frac{3+4+5+2+3+4}{6} = \frac{21}{6} = 3.5$$

Agglomerative Algorithm

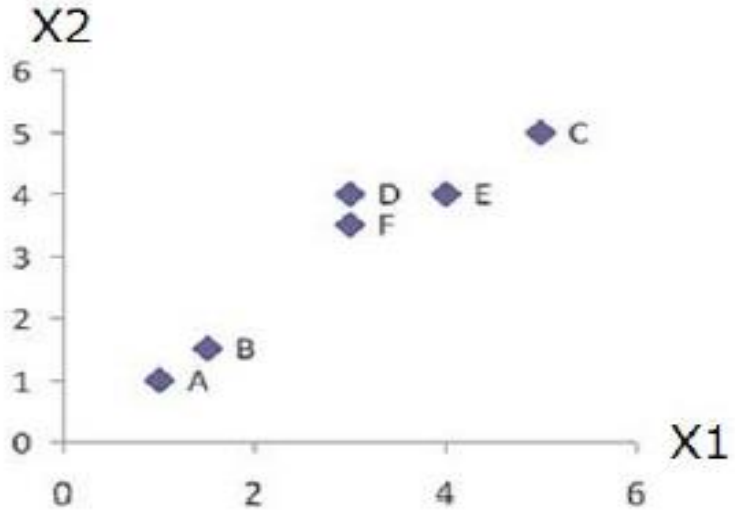
The *Agglomerative* algorithm is carried out in three steps:

1) Convert all object features into a distance matrix
2) Set each object as a cluster (thus if we have *N* objects, we will have *N* clusters at the beginning)
3) Repeat until number of cluster is one (or known # of clusters)
   - Merge two closest clusters
   - Update "distance matrix"



Example

Problem: clustering analysis with agglomerative algorithm



	X1	X2
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5

data matrix

$$d_{AB} = \left((1-1.5)^2 + (1-1.5)^2\right)^{\frac{1}{2}} = \sqrt{\tfrac{1}{2}} = 0.7071$$

$$d_{DF} = \left((3-3)^2 + (4-3.5)^2\right)^{\frac{1}{2}} = 0.5$$

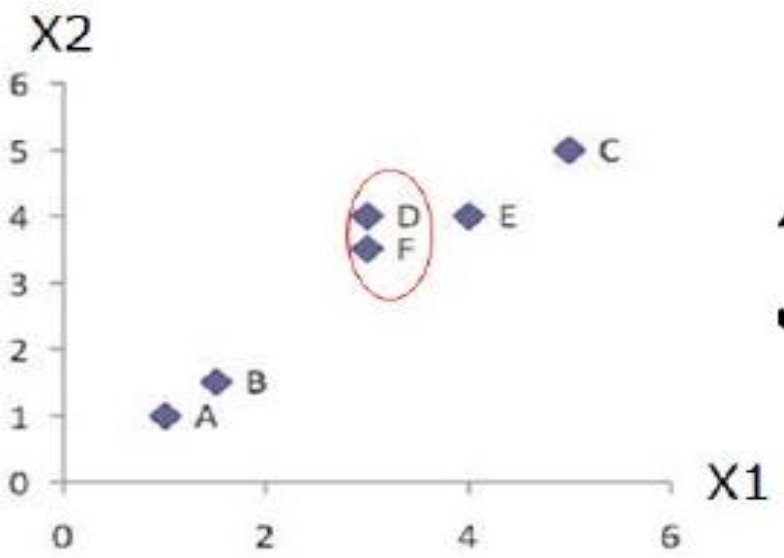
Euclidean distance

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

distance matrix

Example

Merge two closest clusters (iteration 1)



Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	?	4.24
B	0.71	0.00	4.95	?	3.54
C	5.66	4.95	0.00	?	1.41
D, F	?	?	?	0.00	?
E	4.24	3.54	1.41	?	0.00

- Update distance matrix (iteration 1)

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

$$d_{(D,F)\to A} = \min(d_{DA}, d_{FA}) = \min(3.61, 3.20) = 3.20$$

$$d_{(D,F)\to B} = \min(d_{DB}, d_{FB}) = \min(2.92, 2.50) = 2.50$$

$$d_{(D,F)\to C} = \min(d_{DC}, d_{FC}) = \min(2.24, 2.50) = 2.24$$

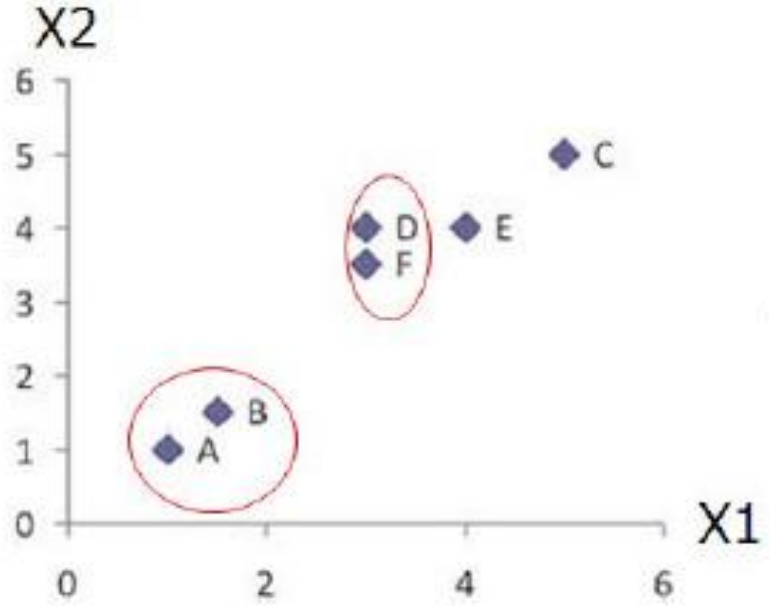
$$d_{E\to(D,F)} = \min(d_{ED}, d_{EF}) = \min(1.00, 1.12) = 1.00$$

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	?	4.24
B	0.71	0.00	4.95	?	3.54
C	5.66	4.95	0.00	?	1.41
D, F	?	?	?	0.00	?
E	4.24	3.54	1.41	?	0.00

Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	**3.20**	**2.50**	**2.24**	0.00	1.00
E	4.24	3.54	1.41	**1.00**	0.00

- Merge two closest clusters (iteration 2)



Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	**3.20**	**2.50**	**2.24**	0.00	1.00
E	4.24	3.54	1.41	**1.00**	0.00

Dist	A,B	C	(D, F)	E
A,B	0	?	?	?
C	?	0	2.24	1.41
(D, F)	?	2.24	0	1.00
E	?	1.41	1.00	0

- Update distance matrix (iteration 2)

Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	**3.20**	**2.50**	**2.24**	0.00	1.00
E	4.24	3.54	1.41	**1.00**	0.00

$$d_{C\to(A,B)} = \min\left(d_{CA}, d_{CB}\right) = \min\left(5.66, 4.95\right) = 4.95$$

$$d_{(D,F)\to(A,B)} = \min\left(d_{DA}, d_{DB}, d_{FA}, d_{FB}\right) = \min\left(3.61, 2.92, 3.20, 2.50\right) = 2.50$$

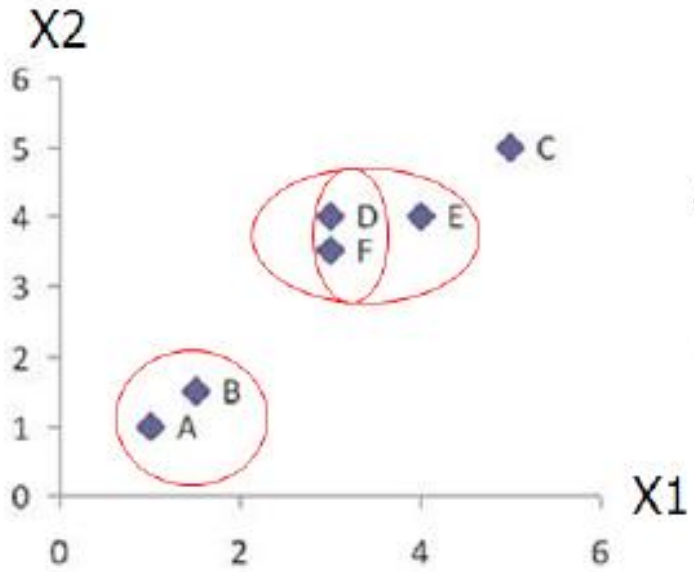
$$d_{E\to(A,B)} = \min\left(d_{EA}, d_{EB}\right) = \min\left(4.24, 3.54\right) = 3.54$$

Dist	A,B	C	(D, F)	E
A,B	0	?	?	?
C	?	0	2.24	1.41
(D, F)	?	2.24	0	1.00
E	?	1.41	1.00	0

Min Distance (Single Linkage)

Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	**4.95**	0	2.24	1.41
(D, F)	**2.50**	2.24	0	1.00
E	**3.54**	1.41	1.00	0

- Merge two closest clusters/update distance matrix (iteration 3)



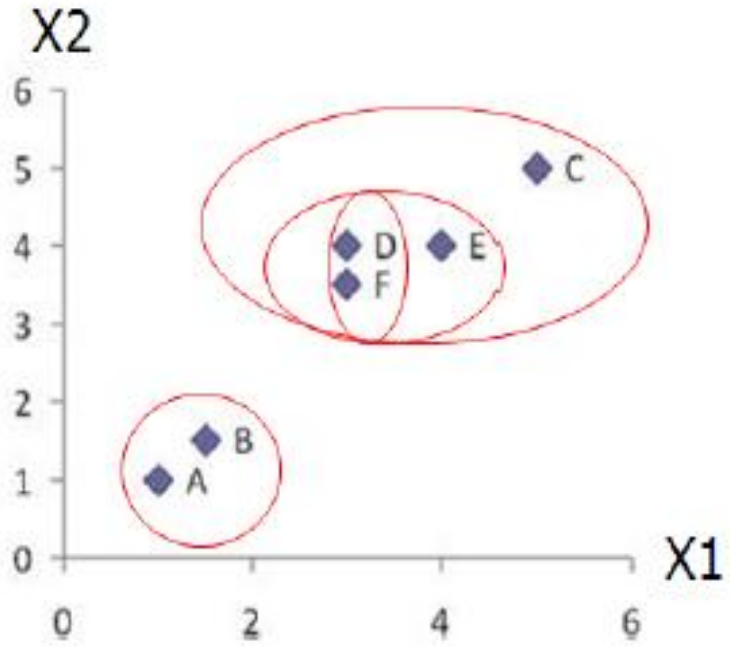
Min Distance (Single Linkage)

Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	**4.95**	0	2.24	1.41
(D, F)	**2.50**	2.24	0	1.00
E	**3.54**	1.41	1.00	0

Min Distance (Single Linkage)

Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	**2.50**	**1.41**	0.00

- Merge two closest clusters/update distance matrix (iteration 4)



Min Distance (Single Linkage)

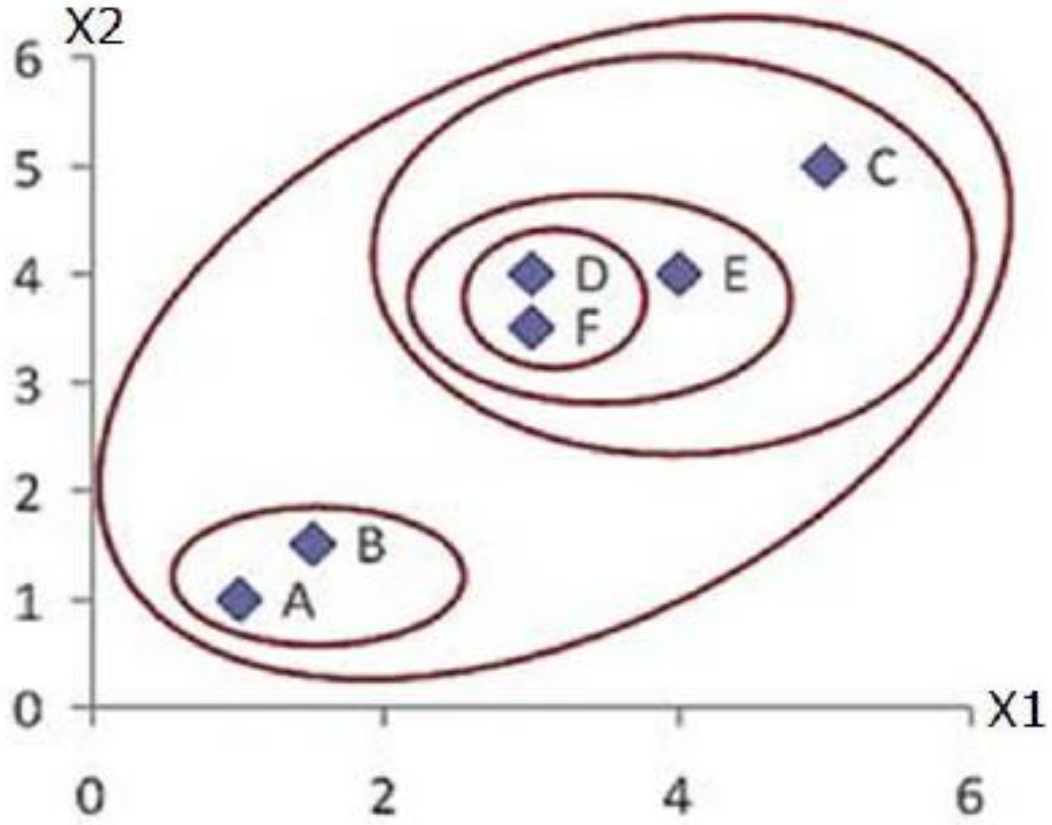
Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	**2.50**	**1.41**	0.00

Min Distance (Single Linkage)

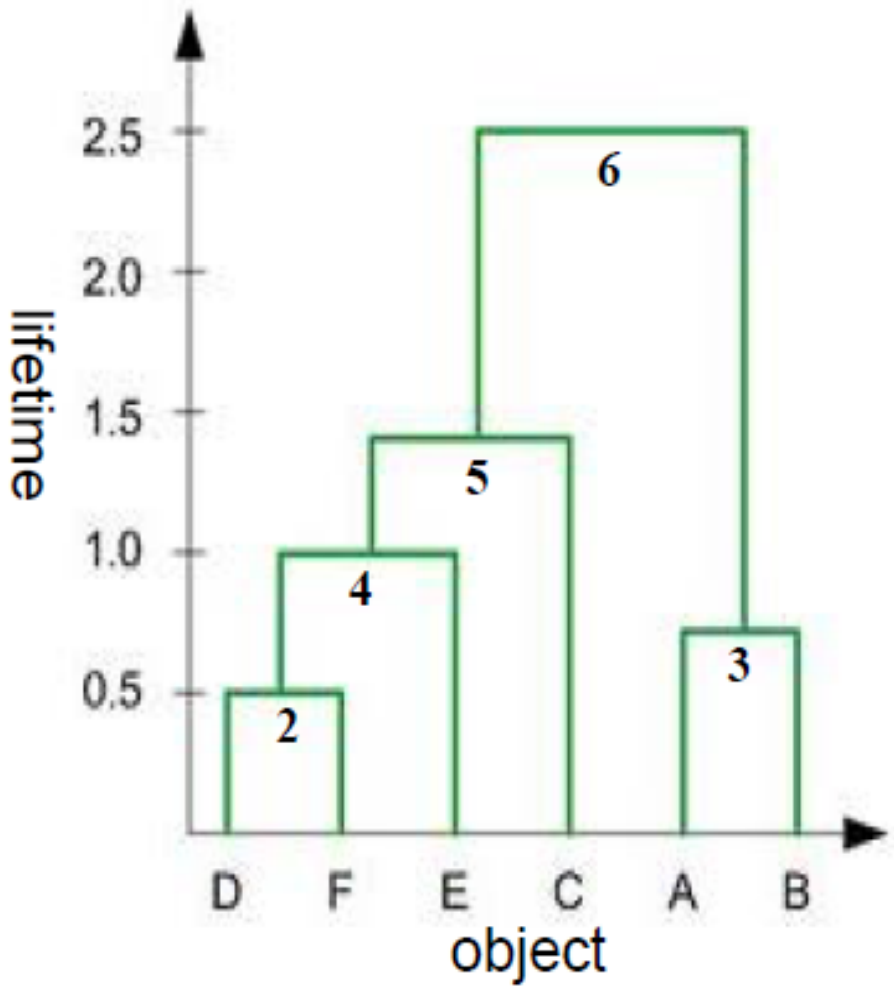
Dist	(A,B)	((D, F), E),C
(A,B)	0.00	2.50
((D, F), E),C	**2.50**	0.00

- Final result (meeting termination condition)

	X1	X2
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5



- Dendrogram tree representation



1. In the beginning we have 6 clusters: A, B, C, D, E and F
2. We merge clusters D and F into cluster (D, F) at distance 0.50
3. We merge cluster A and cluster B into (A, B) at distance 0.71
4. We merge clusters E and (D, F) into ((D, F), E) at distance 1.00
5. We merge clusters ((D, F), E) and C into (((D, F), E), C) at distance 1.41
6. We merge clusters (((D, F), E), C) and (A, B) into ((((D, F), E), C), (A, B)) at distance 2.50
7. The last cluster contain all the objects, thus conclude the computation

It will be Continued….